

README — Text sampling pipeline

A small toolkit to build a corpus from Wikipedia (or your own files) and draw a **traceable sample** of a word, sentence, or document from it, keeping a record of every decision so the whole process can be audited.

It implements the first phases of a methodological pathway for using language models in behavioral science: **load a corpus** → **choose what to observe** → **sample it carefully** → **keep the context around each find**.

There are four code files plus an interactive program (the **exe**):

File	What it is for
<code>corpus_loading.py</code>	Gets the texts and puts them all in one common shape.
<code>sampling.py</code>	Finds and samples words / sentences / documents.
<code>tracelog.py</code>	Keeps the record of every decision (the audit trail).
<code>demo.py</code> (the exe)	The interactive program that asks you questions and runs everything.

To run it: put the four files in one folder and run `python demo.py`. You need Python 3, plus `requests` (for the Wikipedia web API) and `datasets` (for the large-scale Wikipedia option): `pip install requests datasets`.

1. `corpus_loading.py` — getting the texts

Motivation. Before analyzing anything, you need a pile of texts, all stored the same way no matter where they came from. This file is the single front door: it brings texts in and gives them one common shape.

The common shape. Every document is stored as a name, its text (`body`), and some notes about it (`meta`), such as source and language):

```
{ "Article title": { "body": "the text...", "meta": {"source": "wikipedia"} } }
```

Functions

- `make_document(body, meta)` — Builds one document in the common shape. It is the *only* way documents are created, so they all look the same.
- `validate_corpus(corpus)` — Checks a corpus is well-formed (every item has a text, ids are names, etc.). Used as a quick safety check.

- `save_corpus(corpus, path)` — Saves the corpus to a file, with a small summary at the top showing how many articles it has.
- `load_corpus(path)` — Reads a corpus back from a saved file.

Path 1 — Wikipedia web API (a few specific articles):

- `fetch_wikipedia_article(title)` — Downloads one Wikipedia article. If Wikipedia says "too many requests" (error 429), it waits and tries again instead of crashing.
- `load_from_wikipedia(titles)` — Downloads a list of articles by title, pausing briefly between them so Wikipedia doesn't block you.
- `search_wikipedia_titles(term)` — Asks Wikipedia which article titles mention a term.
- `search_and_load_wikipedia(term)` — Combines the two above: searches for articles about a term and downloads them.

Path 2 — your own files:

- `load_from_files(folder)` — Reads every `.txt` file in a folder and turns each into a document. You can attach extra notes per file.

Path 3 — large-scale Wikipedia sampling (the recommended one):

- `sample_wikipedia_dataset(term)` — The powerful option. Instead of asking Wikipedia's servers one article at a time (slow, gets blocked), it streams a full downloaded copy of Wikipedia and **randomly samples** `k` articles that contain your term. It also filters out junk: disambiguation pages ("X may refer to..."), lists, and trailing sections like "See also" and "References", keeping only the real article body.
 - Helpers it uses: `_clean_body` (cuts off the appendix sections) and `_is_disambiguation` (spots disambiguation pages by title or content).

2. `sampling.py` — finding and sampling

Motivation. Once you have a corpus, you need to (a) decide *what* counts as your observation — a word, a sentence, or a whole document — and (b) decide *how* to pick a sample of them. This file separates those two jobs cleanly, so the same picking methods work for any kind of observation.

A) Choosing what to observe — the "extractors"

These turn a corpus into a list of findings. Every finding remembers which document it came from and its exact position (its **offset**), which matters later for matching it to a model.

- `build_pattern(word, mode)` — Prepares the search rule for a word. Three modes: `exact` (only the whole word), `variants` (also short endings like plurals), `loose`

(anywhere, even inside another word). Always ignores upper/lower case.

- `extract_segments(corpus, word)` — Finds every appearance of a word, each with its exact position.
- `extract_sentences(corpus)` — Splits each text into sentences and returns each one as a finding.
- `extract_documents(corpus)` — Returns each whole document as one finding.
- `select_documents_with_word(corpus, word)` — Returns the documents that contain the word (used as the first step of two-stage sampling).

B) Choosing how to pick — the "strategies"

These take any list of findings and return a smaller sample. They don't care whether the findings are words, sentences, or documents.

- `exhaustive(units)` — Keeps everything (no sampling).
- `random_simple(units, n)` — Picks `n` at random, all equally likely.
- `stratified(units, n, key)` — Picks `n` while keeping the proportions of groups (for example, the same share of each source).
- `reservoir(stream, n)` — Picks `n` at random from a huge or streaming list without loading it all at once.

Controls (keeping the sample balanced)

- `cap_per_group(units, max, key)` — Limits how many findings each group (e.g. each article) contributes, so one article doesn't dominate.
- `cap_per_group_spaced(units, max, key, min_distance)` — Same, but the findings it keeps from one article are spread out (at least `min_distance` characters apart), so they come from different parts of the text. If the article is too short to space them out, it relaxes the distance gradually so you still get the number you asked for.
- `summary_per_group(units, key)` — Counts how many findings each group gave, to check the spread.

Context window (KWIC) — the text around the find

- `extract_window(corpus, occurrence, mode, size)` — Adds the surrounding text to a finding: the `left` and `right` of the word. You can measure the window in `chars`, `words`, the whole `sentence`, or the whole `paragraph`. This is what turns a bare list of words into a readable concordance.

Saving (the audit trail for results)

- `save_units(units, path)` — Saves the findings to a file, with a count summary at the top.

- `load_units(path)` — Reads them back.

Orchestrators (putting it together)

- `sample(corpus, unit, strategy)` — Does one extraction plus one strategy in a single call, saves the result, and records the decision.
 - `stage1_documents(corpus, word)` — **Stage 1** of segment sampling: picks the articles that contain the word and saves that sub-corpus.
 - `stage2_occurrences(subcorpus, word)` — **Stage 2**: finds the word's occurrences inside those articles, caps and spaces them, and adds the context window. This is your final sample.
 - `full_flow(corpus, word)` — Runs both stages in a row, allowing a different strategy in each.
 - `print_kwic(concordance)` — Prints the concordance neatly, each word centered with its context on both sides.
-

3. `tracelog.py` — the audit trail

Motivation. The whole point of the method is that every decision is written down with its reason, so the work can be checked and reproduced. This file keeps that record.

- `TraceLog.record(step, operation, ...)` — Adds one entry: which phase, what was done, with which settings, **why** (a reason you write), a short summary of the result, the file it was saved to, and the time.
 - `TraceLog.entries()` — Returns all recorded entries.
 - `TraceLog.save(path)` — Saves the whole record to a file.
-

4. The exe (`demo.py`) — the interactive program

Motivation. This is the friendly front end. It asks you simple questions, runs the right parts of the pipeline based on your answers, and saves everything neatly into its own folder so different runs never overwrite each other.

Each run is saved in `samples/<your-term>_<unit>/`. For example, sampling the word "china" goes to `samples/china_segment/`; a later run on "north korea" goes to `samples/north_korea_segment/`.

What it produces in that folder

File	Contents
<code>corpus_full.json</code>	Every article that was loaded.
<code>stage1_subcorpus.json</code>	The articles that passed stage 1 (contain your word).
<code>stage2_kwic.json</code> (or <code>occurrences_kwic.json</code>)	The final sample: each find with its context.
<code>tracelog.json</code>	The record of every decision and why.

Its helper functions

- `ask`, `ask_int`, `choose` — Ask you a question on screen; pressing Enter accepts the value shown in `[brackets]`.
- `slugify` — Turns your term into a safe folder name ("North Korea" → "north_korea").
- `get_corpus` — Asks where the corpus comes from and builds it; calls one of `_corpus_dataset`, `_corpus_api`, or `_corpus_manual`.
- `ensure_minimum_wikipedia` — If you used the web API and there aren't enough articles with your word, it searches Wikipedia for more.
- `save_corpus_and_show` — Saves the loaded corpus and prints a preview.
- `add_reference` — Attaches a readable reference (article title, source) to each find.
- `choose_strategy` — Asks which sampling strategy you want (and only offers "stratified" when there is more than one source to group by).
- `case_segment` — Runs the word case, in one or two stages (`_segment_two_stages` / `_segment_one_stage`), then reports.
- `case_sentence_or_doc` — Runs the sentence or document case.
- `filter_source` — Keeps only documents from a chosen source.
- `main` — The conductor: asks all the questions in order and runs the rest.

Every question the exe asks you, in order

1. Which unit do you want to sample? — `segment` (a word), `sentence`, or `document`.
2. The target:
 - If segment: **Word/segment to search** (e.g. "yellow"), then **Match type** (`exact` / `variants` / `loose`).
 - If sentence: **Expression the sentence must contain** (empty = all).
 - If document: **Expression the document must contain** (empty = all).

3. **Where does the corpus come from?** — `wikipedia` or `manual`.
 - If Wikipedia: **How should articles be fetched?** — `dataset` (recommended, large-scale) or `api` (a few articles, quick test).
 - **Language code** — `en` or `es`.
 - If dataset: **How many articles to sample (k)?, Minimum term occurrences per article, How many articles to scan at most?, Random seed.**
 - If api: **How many articles to fetch at most?**
 - If manual: you paste each text and its source, ending with a blank line.
4. **Restrict to one source?** — only asked if your corpus mixes several sources; otherwise skipped.
5. **Limits:**
 - **Minimum articles** — the smallest acceptable number of articles.
 - **Maximum articles** (0 = no cap).
 - **Minimum occurrences** — smallest acceptable number of finds.
 - **Maximum occurrences** (0 = no cap).
 - **Maximum occurrences per article** (0 = no cap) — caps how many finds each article contributes.
 - **Random seed** — same number = same result next time (reproducible).
6. **Sampling approach** (segment only): **two stages** (choose articles, then occurrences) or **one stage** (all occurrences at once).
7. **Strategy:** which sampling method to use — `exhaustive`, `random`, `reservoir`, and `stratified` if there are several sources. In two stages you choose one for articles and one for occurrences. If you pick a strategy that needs a number and you didn't set a cap, it asks for that number.
8. **Context window** (segment only): `chars`, `words`, `sentence`, or `paragraph`; and the **Window size** if you chose chars or words.
 - The **minimum distance between occurrences** is then set **automatically** from your window (so the contexts don't overlap) — it is not asked, only reported.

At the end, the exe prints the **TraceLog**: a numbered list of every step it took, with the reason, a summary, and the file where each result was saved.

Quick mental model

```
LOAD a corpus          (corpus_loading.py)
  ↓ all texts, same shape, with notes
CHOOSE what to observe (sampling.py: extractors)
  ↓ words / sentences / documents, each with its position
SAMPLE it              (sampling.py: strategies + controls)
  ↓ a balanced, traceable selection
```

KEEP the context (sampling.py: window)

↓ each find with the text around it

RECORD everything (tracelog.py) → saved per run by the exe (demo.py)